

Лабораторная работа №6

Библиотека TensorFlow (Python)

Что такое TensorFlow

Фреймворк TensorFlow — это относительно простой инструмент, который позволяет быстро создавать нейросети любой сложности. Он очень дружелюбен для начинающих, потому что содержит много примеров и уже готовых моделей машинного обучения, которые можно встроить в любое приложение. А продвинутым разработчикам TensorFlow предоставляет тонкие настройки и API для ускоренного обучения.

TensorFlow поддерживает несколько языков программирования. Главный из них — это Python. Кроме того, есть отдельные пакеты для C/C++, Golang и Java. А ещё — форк TensorFlow.js для исполнения кода на стороне клиента, в браузере, на JavaScript.

Этим возможности фреймворка TensorFlow не ограничиваются. Библиотеку также можно использовать для обучения моделей на смартфонах и умных устройствах (TensorFlow Lite) и создания корпоративных нейросетей (TensorFlow Extended).

Чтобы создать простую нейросеть на TensorFlow, достаточно понимать несколько основных принципов:

- что такое машинное обучение;
- как обучаются нейросети и какие методы для этого используются;
- как весь процесс обучения выглядит в TensorFlow.

Что такое машинное обучение

В обычном программировании всё работает по заранее заданным инструкциям. Разработчики их прописывают с помощью выражений, а компьютер строго им подчиняется. В конце выполнения компьютер выдаёт результат.

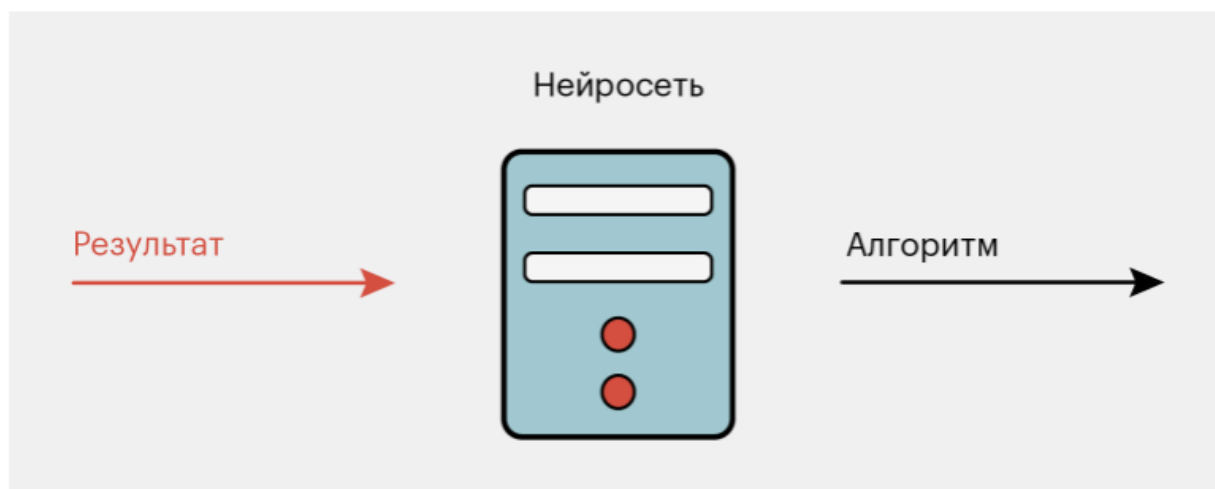
Например, если описать в обычной программе, как вычисляется площадь квадрата, компьютер будет строго следовать инструкции и всегда выдавать стабильный результат. Он не начнёт придумывать новые методы вычисления и не будет пытаться оптимизировать сам процесс вычисления. Он будет всегда следовать правилам — тому самому алгоритму, выраженному с помощью языка программирования.



Обычное программирование — это когда входные данные поступают в программу, а она выдаёт результат

Машинное обучение работает по-другому. Нам нужно отдать компьютеру уже готовые результаты и входные данные и сказать: «Найди алгоритм, который сможет сделать из этих входных данных вот эти результаты». Нам неважно, как он будет это делать. Для нас важнее, чтобы результаты были точными.

Ещё мы должны говорить компьютеру, когда он ответил правильно, а когда — неправильно. Это сделает обучение эффективным и позволит нейросети постепенно двигаться в сторону более точных результатов.



В машинном обучении всё работает по-другому: мы отдаём нейросети результат, а она выдаёт нам алгоритм

В целом машинное обучение похоже на обучение обычного человека. Например, чтобы различать обувь и одежду, нам нужно посмотреть на какое-то количество экземпляров обуви и одежды, высказать свои предположения относительно того, что именно сейчас находится перед нами, получить обратную связь от кого-то, кто уже умеет их различать, — и тогда у нас появится алгоритм, как отличать одно от другого. Увидев туфли после успешного обучения, мы сразу сможем сказать, что это обувь, потому что по всем признакам они соответствуют этой категории.

Как установить TensorFlow

Через Python

Чтобы использовать библиотеку TensorFlow на компьютере, её нужно установить через пакетный менеджер PIP.

Открываем терминал и вводим следующую команду:

```
pip install --upgrade pip
```

Мы обновили PIP до последней версии. Теперь скачиваем сам TensorFlow:

```
pip install tensorflow
```

Если всё прошло успешно, теперь вы можете подключать TensorFlow в Python-коде у вас на компьютере с помощью команды:

```
import tensorflow as tf
```

Но если возникли какие-то ошибки, можете прочитать более подробный гайд на [официальном сайте TensorFlow](#) и убедиться, что у вас скачаны все нужные пакеты.

Как создать нейросеть

Допустим, у нас есть два набора чисел X и Y:

```
X: -1 0 1 2 3 4
Y: -4 1 6 11 16 21
```

Мы видим, что их значения связаны по какому-то правилу. Это правило: $Y = 5X + 1$. Но чтобы компьютер это понял, ему нужно научиться сопоставлять входные данные — X — с результатом — Y. У него сначала могут получаться странные уравнения типа: $2X - 5$, $8X + 1$, $4X + 2$, $5X - 1$. Но, обучившись немного, он найдёт наиболее близкую к исходной формулу.

Обучается нейросеть итеративно — или поэтапно. На каждой итерации она будет предлагать алгоритм, по которому входные значения сопоставляются с результатом. Затем она проверит свои предположения, вычислив все входные данные по формуле и сравнив с настоящими результатами. Так она узнает, насколько сильно ошиблась. И уже на основе этих ошибок скорректирует формулу на следующей итерации.

Количество итераций ограничено разве что временем разработчика. Главное — чтобы нейросеть на каждом шаге улучшала свои предположения, иначе весь процесс обучения будет бессмысленным.

Теперь давайте создадим модель, которая научится решать поставленную выше задачу. Сперва подключим необходимые зависимости:

```
import tensorflow as tf
import numpy as np
from tensorflow import keras
```

Первая зависимость — это наша библиотека **TensorFlow**, название которой мы сокращаем до `tf`, чтобы было удобнее её вызывать в программе. **NumPy** — это библиотека для эффективной работы с массивами чисел. Можно было, конечно, использовать и обычные списки, но NumPy будет работать намного быстрее, поэтому мы берём его. И последнее — **Keras**, встроенная в TensorFlow библиотека, которая умеет обучать нейросети.

Теперь создадим самую простую модель:

```
model = tf.keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
```

Разберём код подробнее. **Sequential** — это тип нейросети, означающий, что процесс обучения будет последовательным. Это стандартный процесс обучения для простых нейросетей: в нём она сначала делает предсказания, затем тестирует их и сравнивает с результатом, а в конце — корректирует ошибки.

`keras.layers.Dense` — указывает на то, что мы хотим создать слой в нашей модели.

Слой — это место, куда мы будем складывать нейроны, которые запоминают информацию об ошибках и которые отвечают за «умственные способности» нейросети. **Dense** — это тип слоя, который использует специальные алгоритмы для обучения.

В качестве аргумента нашей нейросети мы передали указания, какой именно она должна быть:

- `units=1` означает, что модель состоит из одного нейрона, который будет запоминать информацию о предыдущих предположениях;
- `input_shape=[1]` говорит о том, что на вход будет подаваться одно число, по которому нейросеть будет строить зависимости двух рядов чисел: X и Y.

Модель мы создали, теперь давайте её скомпилируем:

```
model.compile(optimizer='sgd', loss='mean_squared_error')
```

Здесь появляются два важных для машинного обучения элемента: функция оптимизации и функция потерь. Обе они нужны, чтобы постепенно стремиться к более точным результатам.

Функция потерь анализирует, насколько правильно нейросеть дала предсказание. А функция оптимизации исправляет эти предсказания в сторону более корректных результатов.

Мы использовали стандартные функции для большинства моделей — `sgd` и `mean_squared_error`. `sgd` — это метод оптимизации, который работает на формулах математического анализа. Он помогает скорректировать формулу, чтобы прийти к правильной. `mean_squared_error` — это функция, которая вычисляет, насколько сильно отличаются полученные результаты по формуле, предложенной нейросетью, от настоящих результатов. Эта функция тоже участвует в корректировке формулы.

Теперь давайте зададим наборы данных:

```
xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
ys = np.array([-4.0, 1.0, 6.0, 11.0, 16.0, 21.0], dtype=float)
```

Как видно, это обычные массивы чисел, которые мы передадим модели на обучение:

```
model.fit(xs, ys, epochs=500)
```

Функция `fit` как раз занимается обучением. Она берёт набор входных данных — `xs` — и сопоставляет с набором правильных результатов — `ys`. И так нейросеть обучается в течение 500 итераций — `epochs=500`. Мы использовали 500 итераций, чтобы наверняка прийти к правильному результату. Суть простая: чем больше итераций обучения, тем точнее будут результаты (однако улучшение точности с каждым повтором будет всё меньше и меньше).

На каждой итерации модель проходит следующие шаги:

- берёт весь наш набор входных данных;
- пытается сделать предсказание для каждого элемента;
- сравнивает результат с корректным результатом;
- оптимизирует модель, чтобы давать более точные прогнозы.

```
1/1 [=====] - 0s 13ms/step - loss: 2.3468e-05
Epoch 490/500
1/1 [=====] - 0s 9ms/step - loss: 2.2986e-05
Epoch 491/500
1/1 [=====] - 0s 13ms/step - loss: 2.2515e-05
Epoch 492/500
1/1 [=====] - 0s 10ms/step - loss: 2.2053e-05
Epoch 493/500
1/1 [=====] - 0s 14ms/step - loss: 2.1599e-05
Epoch 494/500
1/1 [=====] - 0s 14ms/step - loss: 2.1157e-05
Epoch 495/500
1/1 [=====] - 0s 11ms/step - loss: 2.0721e-05
Epoch 496/500
1/1 [=====] - 0s 11ms/step - loss: 2.0295e-05
Epoch 497/500
1/1 [=====] - 0s 10ms/step - loss: 1.9879e-05
Epoch 498/500
1/1 [=====] - 0s 8ms/step - loss: 1.9471e-05
Epoch 499/500
1/1 [=====] - 0s 9ms/step - loss: 1.9070e-05
Epoch 500/500
1/1 [=====] - 0s 10ms/step - loss: 1.8678e-05
<keras.callbacks.History at 0x7f0e82db5ed0>
```

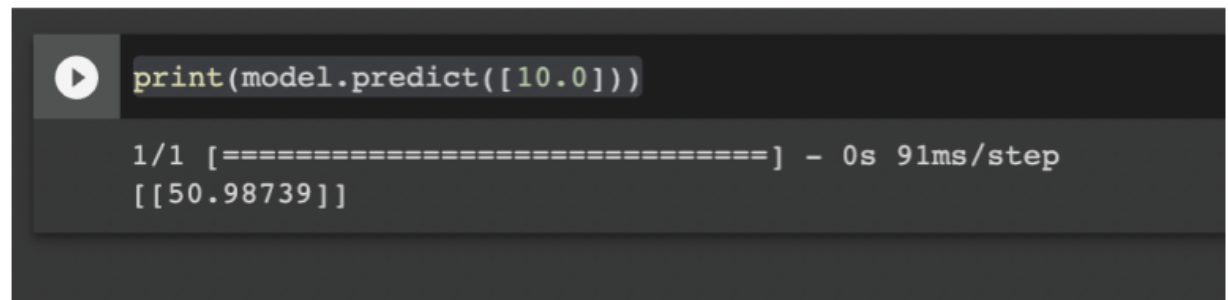
После того как вы запустите предыдущую строчку кода, модель начнёт обучение

Можно заметить, что на каждой итерации TensorFlow выводит, насколько нейросеть сильно ошиблась — `loss`. Если это число уменьшается, то есть стремится к нулю, значит, она действительно обучается и с каждым шагом улучшает свои прогнозы.

Теперь давайте что-нибудь предскажем и поймём, насколько точно наша нейросеть обучилась:

```
print(model.predict([10.0]))
```

Мы вызываем у модели метод `predict`, который получает на вход элемент для предсказания. Результат будет таким:

A screenshot of a Jupyter Notebook cell. The top part shows a code cell with the command `print(model.predict([10.0]))`. Below the code cell, the output is displayed: `1/1 [=====] - 0s 91ms/step` followed by `[ [50.98739] ]`.

```
print(model.predict([10.0]))
```

1/1 [=====] - 0s 91ms/step
[[50.98739]]

Модель сделала предсказание, но оно оказалось не совсем точным

Получилось странно — мы ожидали, что будет число 51 (потому что подставили 10 в выражение $5X + 1$) — но на выходе нейросеть выдала число 50.98739. А всё потому, что модель нашла очень близкую, но не до конца точную формулу — например, $4.891X + 0.993$. Это одна из особенностей машинного обучения.

А ещё многое зависит от выбранного метода оптимизации — то есть того, как нейросеть корректирует формулу, чтобы прийти к нужным результатам. В библиотеке TensorFlow можно найти разные способы оптимизации, и на выходе каждой из них результаты могут различаться. Однако эта тема выходит за рамки нашей статьи — здесь уже необходимо достаточно глубоко погружаться в процесс машинного обучения и разбираться, как именно устроена оптимизация.

Если вы вдруг подумали, что можно просто увеличить число итераций и точность станет выше, то это справедливо лишь отчасти. У каждого метода оптимизации есть своя точность, до которой нейросеть может прийти. Например, она может вычислять результат с точностью до 0.00000001, однако абсолютно верным и точным результат не будет никогда. А значит, и абсолютно точного значения формулы мы никогда не получим — просто из-за погрешности вычислений и особенности функционирования компьютеров. Но если условно установить число итераций в миллиард, можно получить примерно такую формулу:

4.9999999999997X + 0.9999999999991

Она очень близка к настоящей, хотя и не равна ей. Поэтому математики и специалисты по машинному обучению решили, что будут считать две формулы равными, если значения их вычислений меньше, чем заранее заданная величина погрешности — например, 0.0000001. И если мы подставим в формулу выше и в настоящую вместо X число 5, то получим следующее:

$$5 \cdot 5 + 1 = 26$$

$$4.9999999997 \cdot 5 + 0.9999999991 = 25.9999999976$$

Если мы из первого числа вычтем второе, то получим:

$$26 - 25.9999999976 = 0.0000000024$$

А так как изначально мы сказали, что два числа будут равны, если разница между ними меньше 0.0000001, то обе формулы могут считаться идентичными, потому что получившаяся у нас на практике погрешность 0.0000000024 меньше допустимого значения, о котором мы договорились, — то есть 0.0000001. Вот такая интересная математика.

Что запомнить

- Библиотека TensorFlow — это инструмент для создания и обучения нейросетей. При этом вам не нужно углублённо знать высшую математику, чтобы писать простые модели.
- Машинное обучение — это когда мы даём компьютеру входные данные и результаты и просим его понять между ними зависимость. Нам неважно, как он до этого додумается. Главное — точность.
- Обучение нейросети проходит в три этапа: подготовка данных, создание и компиляция модели, само обучение.

 редактировать  fork  скачать

 copy

```
1. import tensorflow as tf
2. import numpy as np
3. from tensorflow import keras# your code goes here
4. model = tf.keras.Sequential([keras.layers.Dense(units=1, input_shape=[1])])
5. model.compile(optimizer='sgd', loss='mean_squared_error')
6. xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
7. ys = np.array([-4.0, 1.0, 6.0, 11.0, 16.0, 21.0], dtype=float)
8. model.fit(xs, ys, epochs=500)
9. print(model.predict([10.0]))
```

Успешно #stdin #stdout #stderr 1.58s 219628KB

 [comments \(?\)](#)

 stdin copy

Standard input is empty

⚙️ stdout

 copy

Epoch 1/500

```
6/6 [=====] - 0s 14ms/sample - loss: 246.4784
Epoch 2/500
```

```
6/6 [=====] - 0s 139us/sample - loss: 194.0327
Epoch 3/500
```

```
6/6 [=====] - 0s 147us/sample - loss: 152.7686
Epoch 4/500
```

```
6/6 [=====] - 0s 181us/sample - loss: 120.3016
Epoch 5/500
```

```
6/6 [=====] - 0s 132us/sample - loss: 94.7559
Epoch 6/500
```

```
6/6 [=====] - 0s 107us/sample - loss: 74.6556
Epoch 7/500
```

```
6/6 [=====] - 0s 109us/sample - loss: 58.8394
Epoch 8/500
```

[illegible]

```
6/6 [=====] - 0s 103us/sample - loss: 2.4919e-05
Epoch 488/500

6/6 [=====] - 0s 100us/sample - loss: 2.4406e-05
Epoch 489/500

6/6 [=====] - 0s 102us/sample - loss: 2.3904e-05
Epoch 490/500

6/6 [=====] - 0s 98us/sample - loss: 2.3413e-05
Epoch 491/500

6/6 [=====] - 0s 96us/sample - loss: 2.2933e-05
Epoch 492/500

6/6 [=====] - 0s 100us/sample - loss: 2.2461e-05
Epoch 493/500

6/6 [=====] - 0s 105us/sample - loss: 2.1999e-05
Epoch 494/500

6/6 [=====] - 0s 101us/sample - loss: 2.1549e-05
Epoch 495/500

6/6 [=====] - 0s 99us/sample - loss: 2.1106e-05
Epoch 496/500

6/6 [=====] - 0s 98us/sample - loss: 2.0673e-05
Epoch 497/500

6/6 [=====] - 0s 98us/sample - loss: 2.0248e-05
Epoch 498/500

6/6 [=====] - 0s 103us/sample - loss: 1.9832e-05
Epoch 499/500

6/6 [=====] - 0s 98us/sample - loss: 1.9425e-05
Epoch 500/500

6/6 [=====] - 0s 98us/sample - loss: 1.9025e-05
[[50.987274]]
```

Контрольные вопросы и задания

1. Что такое TensorFlow?
2. На решение каких задач ориентирована библиотека TensorFlow?
3. Реализуйте нейросеть, рассмотренную в методических указаниях. Проведите обучение нейросети. Проведите эксперименты с обученной нейронной сетью.